PYTHON KURSU • HAFTA 9 • GÜN 2

NumPy 1: Sayısal Diziler

Ders 18 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- NumPy’nin listelerden farkını ve neden hızlı olduğunu açıklayabileceksiniz.
- np.array oluşturup dtype, shape ve ndim özelliklerini okuyabileceksiniz.
- Döngü yazmadan vektörel işlemler ve karşılaştırmalar yapabileceksiniz.
- sum, mean, max, argmax gibi metotlarla dizileri özetleyebileceksiniz.
- Matris oluşturup satır ve sütun (axis) bazında hesap yapabileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 17'in tekrarı
0:10–1:25	75 dk	Blok 1: neden NumPy • kurulum • np.array, dtype, shape • vektörel işlemler • dizi + dizi • karşılaştırma • evrensel fonksiyonlar
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: toplama metotları • indeksleme • dizi oluşturma • rastgele sayılar • matrisler ve axis • not ve sıcaklık uygulamaları • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 17'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Eksik ne?

```
class Yonetici(Calisan):
    def __init__(self, ad, prim):
        self.prim = prim
```

SORU 2 • Ne denir?

```
for kisi in personel:
    kisi.maas_hesapla()
# her sınıf kendi hesabını yapar
```

SORU 3 • Ne olur?

```
@property
def bakiye(self): ...
hesap.bakiye = 5
```

SORU 4 • Miras mı, içerme mi?

```
Sirket — Calisan
Stajyer — Calisan
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 1: kütüphane üyeleri

```
class Uye:
    limit = 0

    def __init__(self, ad):
        self.ad = ad
        self.kitaplar = []

    def odunc_al(self, kitap):
        if len(self.kitaplar) ⩾ self.limit:
            raise ValueError(f"{self.ad}: en fazla {self.limit} kitap.")
        self.kitaplar.append(kitap)


class Ogrenci(Uye):
    limit = 3


class Ogretmen(Uye):
    limit = 10
```

ANAHTAR

self.limit

odunc_al bir kez yazıldı; her alt sınıf kendi limitini kullanıyor.

BLOK 1 • 0:10–1:25 • 75 DK

Sayılarla toplu çalışmak

15 dk
Neden NumPy?

20 dk
np.array

30 dk
Vektörel işlemler

10 dk
Hatalar

BLOK 1 • NEDEN NUMPY? • 15 DK

Liste ile NumPy dizisi aynı işleme farklı cevap verir

```
import numpy as np

liste = [72, 45, 88, 91]
print(liste * 2)            # liste tekrarlanır
dizi = np.array(liste)
print(dizi * 2)             # her eleman 2 ile çarpılır
print(dizi + 5)
```

```
ÇIKTI
[72, 45, 88, 91, 72, 45, 88, 91]
[144  90 176 182]
[77 50 93 96]
```

Benzetme

Liste bir alışveriş poşetidir: içine her şeyi koyarsınız ama hepsine birden işlem yapamazsınız. NumPy dizisi yumurta kolisidir: hepsi aynı türden, düzenli dizilmiş; “hepsini ikiye katla” tek hamlede olur.

BLOK 1 • NEDEN NUMPY? • 15 DK

Bir milyon sayının karesi: liste mi, NumPy mu?

```
import time
import numpy as np

sayilar = list(range(1_000_000))
dizi = np.arange(1_000_000)

bas = time.perf_counter()
kareler_liste = [x * x for x in sayilar]
sure_liste = time.perf_counter() - bas

bas = time.perf_counter()
kareler_dizi = dizi * dizi
sure_dizi = time.perf_counter() - bas

print(f"Liste: {sure_liste*1000:.1f} ms · NumPy: {sure_dizi*1000:.1f} ms")
print(f"NumPy yaklaşık {sure_liste / sure_dizi:.0f} kat hızlı")
```

```
ÖRNEK ÇIKTI
Liste: 50.5 ms · NumPy: 6.6 ms
NumPy yaklaşık 8 kat hızlı
```

Süreler her bilgisayarda farklıdır. NumPy döngüyü C dilinde yazılmış hızlı koda yaptırır ve sayıları bellekte yan yana tutar.

BLOK 1 • NEDEN NUMPY? • 15 DK

Kurulum ve içe aktarma

```
# Terminal (sanal ortam etkinken, Ders 10)
python -m pip install numpy
```

```
import numpy as np

print(np.__version__)
print(type(np.array([1, 2, 3])))
```

```
ÇIKTI
2.5.4
<class 'numpy.ndarray'>
```

Sürüm numarası bilgisayarınızda farklı olabilir; 2.x olması yeterli.

BLOK 1 • NP.ARRAY • 20 DK

np.array ve temel özellikleri

```python
import numpy as np

notlar = np.array([72, 45, 88, 91, 60])
print(notlar)
print("dtype:", notlar.dtype)
print("shape:", notlar.shape, "ndim:", notlar.ndim, "size:", notlar.size)
print("İlk not:", notlar[0], "· Son not:", notlar[-1])
```

```
ÇIKTI
[72 45 88 91 60]
dtype: int64
shape: (5,) ndim: 1 size: 5
İlk not: 72 · Son not: 60
```

Özellik	Anlamı	Bu örnekte
`dtype`	elemanların veri tipi	int64 (64 bitlik tamsayı)
`shape`	her boyuttaki eleman sayısı	(5,): tek boyut, 5 eleman
`ndim`	boyut sayısı	1
`size`	toplam eleman sayısı	5

BLOK 1 • NP.ARRAY • 20 DK

Bir dizide bütün elemanlar aynı tiptedir

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([1, 2.5, 3])
c = np.array([1, 2.5, "3"])
print(a.dtype, b.dtype, c.dtype)
print(b)
print(c)
```

```
ÇIKTI
int64 float64 <U32
[1.  2.5 3. ]
['1' '2.5' '3']
```

Dikkat

Listedeki tek bir metin bütün diziyi metne çevirir; artık hesap yapılamaz. CSV'den okunan verilerde sık görülür.

BLOK 1 • VEKTÖREL IŞLEMLER • 30 DK

Dizi ile sayı: işlem her elemana uygulanır

```
import numpy as np

fiyatlar = np.array([45.0, 12.5, 650.0, 7.5])
print(fiyatlar * 1.20)            # %20 KDV dahil
print(fiyatlar - 5)
print(fiyatlar / 2)
print(fiyatlar // 10, fiyatlar % 10)
```

```
ÇIKTI
[ 54.  15. 780.   9.]
[ 40.    7.5 645.    2.5]
[ 22.5    6.25 325.     3.75]
[ 4.  1. 65.  0.] [5.  2.5 0.  7.5]
```

Bütün aritmetik işleçler (+ – * / // % **) aynı şekilde çalışır. Sonuç **yeni** bir dizidir; orijinali değişmez.

BLOK 1 • VEKTÖREL IŞLEMLER • 30 DK

Dizi ile dizi: eleman eleman eşleşir

```python
import numpy as np

vize = np.array([60, 75, 40, 90])
final = np.array([70, 85, 55, 95])
ortalama = vize * 0.4 + final * 0.6
print(ortalama)
print(final - vize)            # her öğrencinin artışı
```

```
ÇIKTI
[66. 81. 49. 93.]
[10 10 15  5]
```

Öğrenci	Vize	Final	Ortalama
1	60	70	66
2	75	85	81
3	40	55	49
4	90	95	93

Aynı sıradaki elemanlar birbiriyle işleme girer: iki dizinin **uzunluğu aynı** olmalı.

BLOK 1 • VEKTÖREL IŞLEMLER • 30 DK

Karşılaştırma: True/False dizisi

```
import numpy as np

ortalama = np.array([66.0, 81.0, 49.0, 93.0])
gecti = ortalama ⩾ 50
print(gecti)
print(gecti.dtype, gecti.sum(), "öğrenci geçti")
print("Geçme oranı: %", gecti.mean() * 100)
```

```
ÇIKTI
[ True  True False  True]
bool 3 öğrenci geçti
Geçme oranı: % 75.0
```

İpucu

True = 1, False = 0 sayılır. Bu yüzden sum() koşulu sağlayanları sayar, mean() oranını verir.

BLOK 1 • VEKTÖREL IŞLEMLER • 30 DK

Evrensel fonksiyonlar: np.sqrt, np.abs, np.round

```
import numpy as np

degerler = np.array([4, -9, 16.7, 25])
print(np.abs(degerler))
print(np.sqrt(np.abs(degerler)).round(2))
print(np.round(degerler / 3, 1))
```

```
ÇIKTI
[ 4.   9.  16.7 25. ]
[2.   3.   4.09 5.  ]
[ 1.3 -3.   5.6  8.3]
```

Fonksiyon	Yaptığı	math karşılığı
np.sqrt	karekök	math.sqrt (tek sayı)
np.abs	mutlak değer	abs
np.round(d, n)	yuvarlama	round
np.exp, np.log	üstel, logaritma	math.exp, math.log

BLOK 1 • SIK YAPILAN HATALAR

NumPy’ye ilk adımda sık hatalar

Hatalı kod	Hata mesajı	Çözüm
`np.array([1,2,3]) + np.array([1,2,3,4])`	`ValueError: operands could not be broadcast together with shapes (3,) (4,)`	uzunluklar eşit olmalı
`[1, 2, 3] + 5 # liste`	`TypeError: can only concatenate list (not "int") to list`	np.array(…) + 5
`np.array(['1', '2']).mean()`	`TypeError: the resolved dtypes are not compatible with add.reduce. Resolved (dtype('<U1'), dtype('<U1'), dtype('<U2'))`	.astype(float)
`notlar[5] # 4 eleman`	`IndexError: index 5 is out of bounds for axis 0 with size 4`	indeks 0–3
`numpy.array(…) # as np ile alındı`	`NameError: name 'numpy' is not defined`	np.array

1:25–1:40 • 15 DK

Mola

Moladan sonra: bir dizideki en yüksek notu ve kimin aldığını tek satırda bulmak

BLOK 2 • 1:40–2:55 • 75 DK

Özetlemek, üretmek, iki boyuta geçmek

15 dk
Özet metotları

15 dk
Dizi üretmek

20 dk
Matrisler ve axis

25 dk
Uygulama ve alıştırmalar

BLOK 2 • ÖZET METOTLARI • 15 DK

Bir haftanın sıcaklıklarını özetlemek

```
import numpy as np

sicaklik = np.array([17.4, 19.8, 18.1, 16.5, 21.2, 22.0, 20.3])
print("Toplam:", sicaklik.sum().round(1), "· Ortalama:", sicaklik.mean().round(2))
print("En düşük:", sicaklik.min(), "· En yüksek:", sicaklik.max())
print("Standart sapma:", sicaklik.std().round(2), "· Medyan:", np.median(sicaklik))
gunler = ["Pzt", "Sal", "Çar", "Per", "Cum", "Cmt", "Paz"]
print("En sıcak gün:", gunler[sicaklik.argmax()], "· En serin gün:", gunler[sicaklik.argmin()])
```

```
ÇIKTI
Toplam: 135.3 · Ortalama: 19.33
En düşük: 16.5 · En yüksek: 22.0
Standart sapma: 1.89 · Medyan: 19.8
En sıcak gün: Cmt · En serin gün: Per
```

BLOK 2 • ÖZET METOTLARI • 15 DK

İndeksleme ve dilimleme: listedeki gibi

```
import numpy as np

sicaklik = np.array([17.4, 19.8, 18.1, 16.5, 21.2, 22.0, 20.3])
print(sicaklik[0], sicaklik[-1])
print(sicaklik[:3])            # ilk üç gün
print(sicaklik[-2:])           # hafta sonu
print(sicaklik[::2])           # gün aşırı
sicaklik[3] = 16.9             # düzeltme
print(sicaklik[2:5])
```

```
ÇIKTI
17.4 20.3
[17.4 19.8 18.1]
[22.  20.3]
[17.4 18.1 21.2 20.3]
[18.1 16.9 21.2]
```

Dilimleme listedeki kuralların aynısı: **[başlangıç:bitiş:adım]**, bitiş dahil değil. Elemanlar yerinde değiştirilebilir.

BLOK 2 • DIZI ÜRETMEK • 15 DK

Hazır dizi üreten fonksiyonlar

```
import numpy as np

print(np.arange(0, 20, 5))          # range gibi
print(np.linspace(0, 1, 5))         # 0-1 arası eşit aralıklı 5 sayı
print(np.zeros(4), np.ones(3))
print(np.full(5, 7))
```

```
ÇIKTI
[ 0  5 10 15]
[0.   0.25 0.5  0.75 1.  ]
[0. 0. 0. 0.] [1. 1. 1.]
[7 7 7 7 7]
```

Fonksiyon	Ne üretir?	Ne zaman?
`np.arange(bas, bit, adim)`	aralıktaki sayılar (bitiş hariç)	sıra numaraları, saatler
`np.linspace(bas, bit, n)`	eşit aralıklı n sayı (bitiş dahil)	grafik eksenleri (Ders 24)
`np.zeros(n), np.ones(n)`	sıfırlar / birler	sonuçları sonradan doldurmak
`np.full(n, deger)`	hepsi aynı değer	başlangıç değerleri

BLOK 2 • DIZI ÜRETMEK • 15 DK

Rastgele diziler: deneme verisi üretmek

```
import numpy as np

rng = np.random.default_rng(42)     # tohum: aynı sonuçlar
zarlar = rng.integers(1, 7, size=10)
print(zarlar)
notlar = rng.integers(0, 101, size=5)
print(notlar, notlar.mean())
boylar = rng.normal(170, 8, size=4).round(1)
print(boylar)
```

```
ÇIKTI
[1 5 4 3 3 6 1 5 2 1]
[53 98 74 76 72] 74.6
[169.9 163.2 177.  176.2]
```

Tohum (seed) neden?

default_rng(42) her çalıştırmada aynı “rastgele” sayıları üretir. Sınıfta herkes aynı sonucu görür; sonuçlar tekrarlanabilir olur.

BLOK 2 • MATRISLER VE AXIS • 20 DK

İki boyutlu dizi: satırlar ve sütunlar

```
import numpy as np

# satırlar: öğrenciler · sütunlar: vize, ödev, final
notlar = np.array([[60, 80, 70],
                   [75, 90, 85],
                   [40, 65, 55],
                   [90, 95, 95]])
print(notlar.shape, notlar.ndim, notlar.size)
print(notlar[1, 2])          # 2. öğrencinin finali
print(notlar[0])             # 1. öğrencinin bütün notları
print(notlar[:, 0])          # bütün öğrencilerin vizesi
```

```
ÇIKTI
(4, 3) 2 12
85
[60 80 70]
[60 75 40 90]
```

	Vize	Ödev	Final
Öğr. 1	60	80	70
Öğr. 2	75	90	85
Öğr. 3	40	65	55
Öğr. 4	90	95	95

BLOK 2 • MATRISLER VE AXIS • 20 DK

axis: hangi yönde özetleyelim?

```python
import numpy as np

notlar = np.array([[60, 80, 70], [75, 90, 85], [40, 65, 55], [90, 95, 95]])
print("Sınav ortalamaları:", notlar.mean(axis=0).round(1))
print("Öğrenci ortalamaları:", notlar.mean(axis=1).round(1))
print("Genel ortalama:", notlar.mean().round(1))
print("Her sınavın en yükseği:", notlar.max(axis=0))
```

```
ÇIKTI
Sınav ortalamaları: [66.2 82.5 76.2]
Öğrenci ortalamaları: [70.  83.3 53.3 93.3]
Genel ortalama: 75.0
Her sınavın en yükseği: [90 95 95]
```

axis	Yön	Sonuç
0	yukarıdan aşağı ↓	her SÜTUN için bir değer
1	soldan sağa →	her SATIR için bir değer
yok	hepsi	tek bir sayı

BLOK 2 • MATRISLER VE AXIS • 20 DK

Matris üretmek: reshape, zeros, eye

```python
import numpy as np

sayilar = np.arange(1, 13)
print(sayilar.reshape(3, 4))
print(np.zeros((2, 3)))
print(np.eye(3, dtype=int))
```

```
ÇIKTI
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

reshape eleman sayısını korumak zorundadır: 12 eleman → 3×4, 2×6, 4×3… ama 5×3 olmaz.

BLOK 2 • UYGULAMA VE ALIŞTIRMALAR • 25 DK

Uygulama 1: sınıf karnesi tek ekranda

```
import numpy as np

ogrenciler = np.array(["Ece", "Mert", "Zeynep", "Can", "Deniz"])
# sütunlar: vize, ödev, final
notlar = np.array([[60, 80, 70], [75, 90, 85], [40, 65, 55], [90, 95, 95], [55, 40, 48]])
agirlik = np.array([0.3, 0.2, 0.5])

donem = (notlar * agirlik).sum(axis=1)
print("Dönem notları:", donem.round(1))
print("Sınıf ortalaması:", donem.mean().round(1))
print("Birinci:", ogrenciler[donem.argmax()], donem.max().round(1))
print("Kalan sayısı:", (donem < 50).sum())
print("Sınav ortalamaları (vize, ödev, final):", notlar.mean(axis=0).round(1))
```

BLOK 2 • UYGULAMA VE ALIŞTIRMALAR • 25 DK

Sınıf karnesinin çıktısı

```
ÇIKTI
Dönem notları: [69.  83.  52.5 93.5 48.5]
Sınıf ortalaması: 69.3
Birinci: Can 93.5
Kalan sayısı: 1
Sınav ortalamaları (vize, ödev, final): [64.  74.  70.6]
```

YAYMA

notlar * agirlik

(5, 3) matrisin her satırı aynı (3,) ağırlık dizisiyle çarpıldı.

SATIR TOPLAMI

sum(axis=1)

Her öğrenci için ağırlıklı toplam = dönem notu.

KONUM

argmax

En yüksek dönem notunun sırası → ad dizisinden isim.

BLOK 2 • UYGULAMA VE ALIŞTIRMALAR • 25 DK

Uygulama 2: haftalık sıcaklık analizi

```
import numpy as np

gunler = np.array(["Pzt", "Sal", "Çar", "Per", "Cum", "Cmt", "Paz"])
en_yuksek = np.array([19.8, 18.1, 16.5, 21.2, 22.0, 20.3, 17.9])
en_dusuk = np.array([13.2, 12.7, 11.9, 14.0, 15.1, 13.8, 12.2])
fark = en_yuksek - en_dusuk
fahrenheit = en_yuksek * 9 / 5 + 32
print("Günlük fark:", fark.round(1))
print("En büyük fark:", gunler[fark.argmax()], fark.max().round(1), "°C")
print("°F:", fahrenheit.round(0))
sicak_gunler = en_yuksek > 20
print("20 °C üstü gün sayısı:", sicak_gunler.sum(), "·", gunler[sicak_gunler])
```

```
ÇIKTI
Günlük fark: [6.6 5.4 4.6 7.2 6.9 6.5 5.7]
En büyük fark: Per 7.2 °C
°F: [68. 65. 62. 70. 72. 69. 64.]
20 °C üstü gün sayısı: 3 · ['Per' 'Cum' 'Cmt']
```

BLOK 2 • SIK YAPILAN HATALAR

Matrislerde sık hatalar ve tuzaklar

Hatalı kod	Hata mesajı	Çözüm
`np.arange(12).reshape(5, 3)`	`ValueError: cannot reshape array of size 12 into shape (5,3)`	satır × sütun = 12
`np.zeros(2, 3)`	`TypeError: Cannot interpret '3' as a data type`	np.zeros((2, 3))
`m[2, 0] # 2 satır var`	`IndexError: index 2 is out of bounds for axis 0 with size 2`	satır indeksi 0–1

Hata vermeyen tuzak	Ne olur?	Doğrusu
`a = np.array([1, 2, 3]); a[0] = 2.7`	a → [2 2 3] (sessizce kesilir)	dtype=float ile oluşturun
`notlar.mean(axis=0) ↔ axis=1`	öğrenci yerine sınav ortalaması	sonucun uzunluğuna bakın

BLOK 2 • ÖZET

Liste mi, NumPy dizisi mi?

	Python listesi	NumPy dizisi
Eleman tipi	karışık olabilir	tek tip (dtype)
liste * 2	tekrarlar	her elemanı 2 ile çarpar
Boyut değiştirme	append ile kolay	sabit boyut; yeni dizi oluşturulur
Hız (büyük sayısal veri)	yavaş	çok hızlı
Matematik	döngü gerekir	vektörel, tek satır
Ne zaman?	karışık, büyüyen koleksiyonlar	sayısal hesap, tablo verisi

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 15 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Fiyat listesi

5 ürün fiyatı: %10 indirimli fiyatlar, toplam tutar, en pahalı ürünün konumu, 100 TL üstü ürün sayısı.

ORTA • 5 dk

2 • Adım sayar

rng = np.random.default_rng(7) ile 30 günlük adım sayısı (2000–15000). Ortalama, en iyi gün (kaçıncı gün?), 10.000 üstü gün oranı.

ZOR • 6 dk

3 • Mağaza × ay

3 mağaza × 4 aylık satış matrisi oluşturun. Her mağazanın toplamı, her ayın ortalaması, en çok satan mağaza.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: fiyat listesi

```
import numpy as np

fiyatlar = np.array([45.0, 120.0, 650.0, 7.5, 99.9])
print("İndirimli:", (fiyatlar * 0.9).round(2))
print("Toplam:", fiyatlar.sum())
print("En pahalı ürünün konumu:", fiyatlar.argmax())
print("100 TL üstü:", (fiyatlar > 100).sum(), "ürün")
```

```
ÇIKTI
İndirimli: [ 40.5  108.   585.     6.75  89.91]
Toplam: 922.4
En pahalı ürünün konumu: 2
100 TL üstü: 2 ürün
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 2: adım sayar

```
import numpy as np

rng = np.random.default_rng(7)
adimlar = rng.integers(2000, 15001, size=30)
print("Ortalama:", adimlar.mean().round(0))
print("En iyi gün:", adimlar.argmax() + 1, "·", adimlar.max(), "adım")
print("10.000 üstü gün oranı: %", ((adimlar > 10000).mean() * 100).round(1))
```

```
ÇIKTI
Ortalama: 8918.0
En iyi gün: 27 · 14876 adım
10.000 üstü gün oranı: % 43.3
```

Çözüm 3: mağaza × ay

```
import numpy as np

magazalar = np.array(["Kadıköy", "Bornova", "Çankaya"])
# sütunlar: Ocak, Şubat, Mart, Nisan (bin TL)
satis = np.array([[120, 135, 128, 150],
                  [ 98, 110, 105, 120],
                  [140, 138, 152, 160]])
print("Mağaza toplamları:", satis.sum(axis=1))
print("Ay ortalamaları:", satis.mean(axis=0).round(1))
print("En çok satan:", magazalar[satis.sum(axis=1).argmax()])
```

```
ÇIKTI
Mağaza toplamları: [533 433 590]
Ay ortalamaları: [119.3 127.7
  128.3 143.3]
En çok satan: Çankaya
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- **np.array** tek tipli, hızlı dizi; **dtype**, **shape**, **ndim**, **size**.
- İşlemler **vektörel**: dizi + 5, dizi * dizi, dizi > 50 → döngü yok.
- Özet metotları: **sum, mean, min, max, std**; konum için **argmax / argmin**.
- Üretmek: **arange, linspace, zeros, ones, full**; rastgele: **default_rng(tohum)**.
- Matriste **m[satır, sütun]**; **axis=0** sütun başına, **axis=1** satır başına.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Kendi notlarınız

En az 5 dersin vize/final notlarıyla (uydurabilirsiniz) bir matris oluşturun; ağırlıklı ortalamalar, en iyi ve en zayıf ders, genel ortalama.

2 • ZORUNLU

Zar istatistiği

default_rng(2026) ile 6000 zar atın. Her yüzün kaç kez geldiğini bulun (== ve sum) ve 1000'e ne kadar yakın olduğunu yorumlayın.

3 • BONUS

Hava API'si + NumPy

Ders 11'deki Open-Meteo fonksiyonuyla 7 günlük tahmini alın; en yüksek/düşük dizileriyle Uygulama 2'yi kendi şehriniz için yapın.

Teslim: ders18_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 10 • GÜN 1

NumPy 2: matris işlemleri ve filtreleme

Bugün dizileri özetledik. Gelecek ders içlerinden seçecek, sıralayacak ve birleştireceğiz:

- Matrislerle dört işlem ve yayma (broadcasting)
- Filtreleme: dizi[dizi > 100], & ve |
- Sıralama: sort, argsort
- Birleştirme: concatenate, vstack • mağaza × ay satışları

Önizleme

```
import numpy as np

satis = np.array([120, 98, 140, 135, 110])
print(satis[satis > 115])
```

```
[120 140 135]
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
NumPy: Absolute beginners	numpy.org/doc/stable/user/absolute_beginners.html
NumPy quickstart	numpy.org/doc/stable/user/quickstart.html
Rastgele sayı üretimi	numpy.org/doc/stable/reference/random
Broadcasting (yayma)	numpy.org/doc/stable/user/basics.broadcasting.html